

SecureRoom — Time-Limited Room & Locker Unlocking System

Version: 1.0

Author: Generated for prazal-tech

Date: 2025-12-05

Contents

- Overview
- Goals & Threat Model
- High-level Architectures
 - Cloud-assisted mode
 - Offline-signed mode
- Cryptographic primitives and hardware
- Token formats (QR & short code)
- Token generation algorithms (server)
- Lock verification logic (firmware)
- Short code design & HMAC derivation
- Replay protection, rate-limiting & logging
- Provisioning, key lifecycle & OTA updates
- Hardware recommendations & integrations
- Example end-to-end snippets
 - Server: token & QR generation (Python)
 - Lock: token verification (Python pseudocode for MCU)
 - Short code derivation (HMAC)
- UI & UX notes (mobile / admin)
- Operational checklist & testing
- How to produce a PDF from this document
- Appendix: further reading & references

Overview

This document describes a practical, secure system for unlocking rooms and lockers using time-limited tokens presented as QR codes or short alphanumeric codes on a phone or application. Two deployment modes are presented:

- Cloud-assisted: server issues tokens and can revoke/log centrally.
- Offline-signed: tokens are signed by a provisioning authority (PA) and verified by the lock without online contact.

The design balances security, ease of use, and real-world constraints (keypad-only locks, camera-enabled locks, limited MCU resources).

Goals & Threat Model

Goals

- Provide time-limited, revocable access tokens for rooms/lockers.
- Support user flow via phone app: display QR or short code.
- Allow locks to verify tokens either online (cloud) or offline (PA-signed).
- Prevent replay attacks, token cloning, and brute force guessing.
- Produce an audit trail for access events.

Threat model (attacker capabilities)

- Network eavesdropping (public Wi-Fi / BLE sniffing).
- Local wireless sniffing and replay of captured tokens.
- Passive observation (someone photographing the QR/code).
- Brute-force attempts on keypad or API.
- Physical access to the lock (no high-end lab extraction unless a secure element is not used).

Adversaries we aim to mitigate:

- Unauthorized unlocking via replay or guessing.

- Token compromise from stolen phones / screenshots.
- Firmware tampering (mitigated by firmware signing).

High-level architectures

1) Cloud-assisted (recommended when you have reliable connectivity)

- Flow:
 - Admin or user requests access via server (web/admin/mobile).
 - Server authenticates requester, creates a short-lived token containing lock_id, expiry, nonce, and optionally user_id.
 - Server returns a token to the mobile app. The app displays the token as a QR or derives a short code to show.
 - Lock (online or via gateway) validates token with server or verifies signature if token is signed by server key.
 - On success: lock opens and logs the event centrally.
- Pros:
 - Immediate revocation and central logging.
 - Short tokens (good UX).
- Cons:
 - Requires server availability or gateway.

2) Offline-signed (recommended for isolated/air-gapped locks)

- Flow:
 - Provisioning Authority (PA) signs tokens (Ed25519). Mobile requests token from server (before arrival).
 - Mobile displays QR or code. Lock verifies sig with PA public key stored during provisioning.
 - On success: lock opens and logs locally (uploads when online).
- Pros:
 - Works without network during unlock.
- Cons:
 - Revocation is harder — use short token lifetimes or CRLs (costly for constrained locks).
 - Need secure provisioning of PA public key into each lock.

Cryptographic primitives & hardware

Recommended primitives

- Signatures: Ed25519 (compact, fast verifications).
- Key agreement (if needed): X25519.
- Symmetric authenticated encryption: ChaCha20-Poly1305 (recommended for constrained MCUs) or AES-256-GCM (if hardware AES present).
- MAC for short codes: HMAC-SHA256.
- Hash: SHA-256.
- RNG: hardware RNG or secure element RNG.

Hardware

- Secure element for private key storage: Microchip ATECC608A, NXP SE050 or equivalent.
- MCU: Cortex-M4/M33 family for performance and support.
- Camera module for QR scanning or keypad for code entry.
- BLE/Wi-Fi module for cloud-assisted operation (ESP32 recommended for prototyping).

Token formats

We recommend compact structured tokens. Prefer binary or CBOR for compactness; JSON base64 is OK for prototypes.

Canonical token fields

- lid: lock identifier (string)
- uid: user identifier (optional, for logging)
- iat: issued-at (Unix timestamp)
- exp: expiry (Unix timestamp)
- nonce: random nonce (≥ 96 bits)
- perms: optional permissions (open, temp-access, maintenance)

Signed token layout (binary recommended)

- token_bytes = canonical_serialization(token_fields) // e.g., CBOR or canonical JSON
- signature = Ed25519_sign(PA_SK, token_bytes)
- payload = base64url(token_bytes || signature)

Short alphanumeric code (human-entered)

- 8 characters, Base36 (0-9, A-Z) recommended for 8 characters → ~41 bits entropy.
- Derived via HMAC from a per-lock secret so the lock can independently verify.
- Bind to token.nonce or token_id and expiry.

Token lifetime guidelines

- QR (machine-scan, higher security): 1–5 minutes.
- Short code (manual entry, lower entropy): 3–10 minutes, plus lock rate limits.
- Guest access (multi-day): issue multiple short-lived tokens or longer-signed tokens with careful revocation.

Token generation algorithms (server)

Design notes

- Use strong entropy for nonce: at least 96 bits / 12 bytes.
- Use canonical serialization for signatures (sorted JSON separators or CBOR).
- Store issued nonce and token_id in server DB for auditing and optional single-use enforcement.

Python example (server-side token generation)

```
```python
```

```
import os, time, json, base64
```

```
from nacl.signing import SigningKey # pip install pynacl
```

```
PA_SK = SigningKey.generate() # In production: load from secure storage
```

```
PA_PK = PA_SK.verify_key
```

```
def make_token(lock_id, user_id=None, ttl_seconds=300):
```

```

now = int(time.time())
exp = now + ttl_seconds
nonce = base64.urlsafe_b64encode(os.urandom(12)).decode().rstrip('=')
token = {"lid": lock_id, "uid": user_id or "", "iat": now, "exp": exp, "nonce": nonce}
tb = json.dumps(token, separators=(',', ':'), sort_keys=True).encode()
sig = PA_SK.sign(tb).signature
payload = base64.urlsafe_b64encode(tb + sig).decode().rstrip('=')
return payload, token
...

```

#### Short code derivation (server)

- If the lock has a per-lock secret  $S_{\text{lock}}$  (32 bytes), derive code:

```
code = base36_truncate(HMAC_SHA256(S_lock, token_nonce || exp)) -> 8 chars
```

#### Lock verification logic (firmware)

High-level steps when a QR or code is presented:

1. Parse presented payload: decide if it's a full signed token (QR) or a short code.
2. If signed token:
  - Decode base64 → token\_bytes || signature.
  - Verify Ed25519 signature with stored PA public key.
  - Parse token fields and check lid == this\_lock\_id.
  - Check current time within [iat - skew, exp + skew].
  - Check nonce has not been used/replayed (track recent nonces).
  - On success: actuate unlock and log event.
3. If short code:
  - Recompute expected code via HMAC with stored  $S_{\text{lock}}$  and compare constant-time.
  - Verify expiry associated with that code (the lock must be able to derive/check expiry; include token id or epoch in derivation).
  - Rate-limit and enforce lockout on repeated failures.
4. Logging:

- Write event: (timestamp, uid if present, token\_id or nonce, success/failure).
- If online, upload; if offline, store in circular log with tamper detection (e.g., signed counters).

Firmware pseudocode (Python-style)

```
```python
def verify_payload(payload_b64):
    raw = base64.urlsafe_b64decode(payload_b64 + '==')
    token_bytes = raw[:-64]
    sig = raw[-64:]
    if not ed25519_verify(PA_PK, token_bytes, sig):
        return False, "invalid signature"
    token = json.loads(token_bytes)
    if token['lid'] != MY_LOCK_ID: return False, "wrong lock"
    now = current_unix_time()
    if token['exp'] < now: return False, "expired"
    if nonce_seen_before(token['nonce']): return False, "replay"
    mark_nonce_seen(token['nonce'])
    unlock()
    log_event('unlock', token.get('uid',''), token['nonce'], now)
    return True, "ok"
```
```

Short code design & HMAC derivation

Parameters

- Lock secret: S\_lock (32 bytes), provisioned securely into lock SE.
- Input: token\_nonce OR small token\_id + epoch bucket.
- Derivation: mac = HMAC\_SHA256(S\_lock, token\_nonce || str(exp))
- Truncate to first 6 bytes → convert to Base36 → take 8 chars.

Example Python HMAC → base36 code

```

` `` python

import hmac, hashlib

def hmac_code(S_lock: bytes, token_nonce: str, exp: int, length=8):

 data = token_nonce.encode() + b'|' + str(exp).encode()

 mac = hmac.new(S_lock, data, hashlib.sha256).digest()

 v = int.from_bytes(mac[:6], 'big')

 alphabet = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ'

 s = ""

 for _ in range(length):

 s = alphabet[v % 36] + s

 v //= 36

 return s

` ``

```

Replay protection, rate-limiting & logging

Replay protection

- Keep a small ring buffer of recent nonces (e.g., last 256). Store nonce and timestamp; reject re-use.
- If lock memory is extremely constrained, store a bloom filter with periodic reset, or require one-time-use tokens (server tracks use).

Rate limiting & lockout

- Allow limited failed attempts: e.g., 5 attempts per minute, then a cooldown (5–15 minutes).
- For critical environments, escalate on repeated attempts (alert to admin).

Logging & audit

- Local log: append events signed with a device-key or stored with monotonic counter for tamper evidence.
- Upload to server whenever online. For offline-only deployments, export logs during maintenance.

## Provisioning, key lifecycle & OTA updates

### Provisioning steps

- During production:
  - Generate device key pair inside Secure Element (private key never leaves).
  - Register device public key and device\_id with PA database.
  - Install PA public key and firmware verification key into device's immutable storage.
  - Generate per-lock secret S\_lock if using HMAC short codes; store securely in SE.
- Admin setup:
  - Admin server stores PA\_SK in HSM or vault (not on developer laptop).
  - Admin UI for issuing tokens and revoking access.

### Revocation & rotation

- Cloud-assisted: revoke tokens centrally and refuse server validations.
- Offline-signed: use short token lifetimes to limit exposure. Optionally distribute CRLs to locks (awkward).
- Key rotation:
  - Include key\_id in tokens so locks can accept tokens signed by multiple valid public keys during rotation.
  - Perform roll forward updates with overlap windows.

### OTA firmware updates

- Bootloader must verify firmware signatures with an immutable public key.
- Firmware images signed by vendor using Ed25519 or RSA (vendor private key stored offline/HSM).
- OTA transport can be encrypted but must be authenticated.

### Hardware recommendations & integrations

### Cameras and code scanning

- For QR-based systems, use either a lock-side camera module or rely on the phone to act as a BLE-to-lock relay.
- Camera must support real-time scanning; consider zxing or libzbar for embedded or gateway scanning.

#### Keypad-only locks

- Use short alphanumeric codes (8 chars) with strong rate limits and one-time usage.

#### Secure element

- ATECC608A recommended: supports ECDSA, ECDH, secure key storage, and hardware RNG.
- Use SE to perform signature verification/ HMAC operations if possible.

#### Networking & connectivity

- Wi-Fi/Cellular gateway or BLE: mobile can relay tokens or the lock can contact server for final validation.
- Time synchronization: locks need reasonably accurate time (NTP via gateway or periodic sync) for expiry validation. For fully offline locks, accept token iat/exp with resilient clock or accept a margin.

#### Example end-to-end snippets

Server: generate QR payload & image (Python)

```
` `` python
```

```
requires: pip install pynacl qrcode
```

```
import qrcode, base64, json, os, time
```

```
from nacl.signing import SigningKey
```

```
PA_SK = SigningKey.generate()
```

```
def generate_payload(lock_id, uid="alice", ttl=300):
```

```
 now=int(time.time())
```

```
 exp=now+ttl
```

```
 nonce=base64.urlsafe_b64encode(os.urandom(12)).decode().rstrip('=')
```

```

token={"lid":lock_id, "uid":uid, "iat":now, "exp":exp, "nonce":nonce}
tb=json.dumps(token, separators=(',', ':'), sort_keys=True).encode()
sig=PA_SK.sign(tb).signature
payload=base64.urlsafe_b64encode(tb+sig).decode().rstrip('=')
return payload, token

```

```

payload, token = generate_payload("LOCK-123")
img = qrcode.make(payload)
img.save("token_qr.png")
print("saved token_qr.png, token:", token)
` ` `

```

Lock: verify payload (Python-style pseudocode)

```

` ` ` python

```

```

import base64, json, time
from nacl.signing import VerifyKey

```

```

PA_PK = VerifyKey(b'...') # Provisioned verify key bytes

```

```

SEEN_NONCES = set()

```

```

MY_LOCK_ID = "LOCK-123"

```

```

def verify_qr(payload_b64):
 raw=base64.urlsafe_b64decode(payload_b64 + '=')
 tb=raw[:-64]; sig=raw[-64:]
 try:
 PA_PK.verify(tb, sig)
 except Exception:
 return False, "invalid signature"
 token=json.loads(tb.decode())
 if token['lid'] != MY_LOCK_ID: return False, "wrong lock"
 now=int(time.time())

```

```
if token['exp'] < now: return False, "expired"
if token['nonce'] in SEEN_NONCES: return False, "replay"
SEEN_NONCES.add(token['nonce'])
unlock()
log_event('unlock', token.get('uid',''), token['nonce'], now)
return True, "ok"
...
```

## UI & UX notes (mobile / admin)

### Mobile app

- Screen: show QR prominently; also show short code (copy button).
- Timer: show countdown to expiry for user confidence.
- Share: allow secure share (SMS/email) only via server-issued token (no sharing of private keys).
- Protect tokens: require authentication to see active token in app (lock screen, biometrics).

### Admin UI (web)

- Issue tokens with selectable expiry (default 5 minutes).
- Show active issued tokens with token\_id, expiry, user, and ability to revoke.
- Audit logs: show unlock events (time, lock\_id, uid).

## Operational checklist & testing

### Testing

- Unit tests for token generation, signature verification, HMAC code derivation.
- Integration tests: mobile -> lock end-to-end flows with expired tokens and replay attempts.
- Penetration testing: attempt brute-force short codes, replay captured QR, and inspect logs.
- Fault testing: network outages, locked storage failures, SE failures.

### Monitoring & alerts

- Alert on repeated failed attempts or brute force patterns.

- Periodic integrity checks of firmware and keys.

How to produce a PDF from this Markdown

Option A (recommended): Use pandoc

1. Install pandoc and a TeX engine (e.g., TeX Live or TinyTeX).

2. Convert:

```
pandoc room-lock-encryption-system.md -o room-lock-encryption-system.pdf --pdf-engine=xelatex -V geometry:margin=1in -V fontsize=11pt
```

Option B: Use VS Code or GitHub

- Open room-lock-encryption-system.md in VS Code and export to PDF using the Markdown preview or extensions.

- Or open GitHub preview and print to PDF.

Appendix: further reading & references

- Ed25519: <https://ed25519.cr.yp.to/>
- ChaCha20-Poly1305: RFC 7539
- HMAC: RFC 2104
- Time-based OTP (if needed): RFC 6238
- Microchip ATECC608A documentation for secure element usage.

Security disclaimers

- Do not use demo keys in production.
- Store server signing keys in an HSM/vault (AWS KMS, HashiCorp Vault).
- Use secure provisioning and enforce firmware signing.
- This document is a design guide; adapt parameters to your threat model and compliance requirements.

Contact / Next steps

If you want, I can:

- Generate the QR PNG samples and a set of demo short codes.

- Produce a ready-to-run repository with server, mobile UI mockups, and a simulated lock firmware.
- Convert this Markdown to a PDF and provide the file (if you can provide a way to receive attachments), or guide you step-by-step to run the pandoc command locally.

---

END